Gmail **Family account <galguerrero@gmail.com>**

Simulator Architecture Roadmap

1 mensaje

Family account <galguerrero@gmail.com> 22 de agosto de 2026 a las 22:17
Para: Family account <galguerrero@gmail.com>

Retr01 Simulator: Architecture & Optimization Roadmap

Building a cycle-accurate hardware simulator is one of the most challenging and rewarding programming tasks in existence. You are translating electrical physics into sequential software.

The code you wrote for the W65C02S proves you have the engineering chops to do this. Do not give up. This document is your playbook. It is split into two parts: the questions you need to answer right now to build the foundation, and the optimization tricks you will use later to make it fast.

Phase 1: Architectural Questions (Answer These First)

Before you write the next chip, you need to define how the "motherboard" coordinates all these independent C structs. Think through these design decisions:

1. The Master Clock: How does time move forward?

Right now, you have an `r01s_w65c02s_tick()` function. How will the main loop call this alongside the 20 MHz ATmega and the 74HC logic gates?

- **The Array Method (Recommended):** Do you have a central `r01s_board_tick()` function that simply iterates over a contiguous array of `R01sEntity*` pointers, calling their `->tick()` functions in sequence?
- **The Base Resolution:** Your fastest chip is the 20 MHz ATmega. Does your master loop tick at 20 MHz? If so, the 8 MHz W65C02S only evaluates its phase every 2.5 ticks. How are you tracking fractional clock dividers?

2. The Bus Resolution: How do you handle electrical propagation?

In the real world, electricity is instantaneous. In C, code runs sequentially. If the W65C02S sets the address bus to `$FE00`, the `74HC138` decoder needs to see that change to pull the Latch `/CE` pin low, which then tells the `74HC573` latch to swallow the data bus.

- **The Two-Pass Method:** Will your main loop have an `Eval` phase (where all chips read their inputs and calculate internal state) followed by a `Drive` phase (where all chips output to the virtual wires)? *This is how you prevent the order of chips in your array from breaking the simulation.*
- **Bus Conflicts:** If two chips accidentally drive the Data Bus at the same time, how does `r01s_bus_write()` handle it? Does it log a warning, or crash the sim to warn you of a hardware bug?

3. State Management: Where do the wires live?

When you call `r01s_entity_sense(e, "BE")`, where is it reading from?

- Do you have a global `Netlist` object that holds the state of every wire?
- Are pins directly passing pointers to other pins? (e.g., CPU Pin 36 physically points to PLD Pin 12).

Phase 2: The Optimization Playbook (How to get 60 FPS)

Right now, your goal is **100% accuracy**. Build it slow. Prove your hardware works. Get a pixel on the screen.

Once it boots, it might run at 3 FPS. *Do not panic.* That is normal. When you are ready for speed, you will execute these four optimization passes.

Pass 1: Kill the Strings (The Low-Hanging Fruit)

String comparisons inside a 20-million-tick-per-second loop will instantly murder your CPU cache.

- **Current:** `r01s_entity_sense(e, "BE")` requires traversing an array and running `strcmp()`.
- **The Fix:** Replace string names with integer Enums or direct array indices.
- **Future:** `r01s_entity_sense(e, PIN_W65C02S_BE)`. This turns an expensive string search into a 1-cycle direct memory lookup.

Pass 2: Bitmask the Buses (The Fast Path)

Currently, `r01s_bus_write(e, "A", 16, c->ab)` abstracts a 16-pin bus. Under the hood, if this is setting 16 individual boolean pin states, it is too slow.

- **The Fix:** Create global bitmasks for major buses. A `uint32_t system_address_bus` variable can hold all 16 pins.
- Instead of iterating 16 pins, the CPU does `system_address_bus = c->ab;` in exactly one CPU cycle. The memory chips simply read `system_address_bus & 0x7FFF`.

Pass 3: The "Super Component" Merge

Once you physically prove that your `$FExx` latches, multiplexers, and PLDs work together flawlessly in the slow simulation, you no longer need to simulate their individual pins!

- **The Fix:** Delete the discrete 74HC C-files. Create a single `motherboard_glue.c` entity.
- Inside its `tick()` function, you write pure C logic: `if (address >= 0xFE00) latch_state = data_bus;`.
- You just replaced the electrical simulation of 15 chips with three lines of C code, increasing your frame rate by a factor of 10.

Pass 4: Data-Oriented Design (DOD)

Modern CPUs (like your PC) are incredibly fast, but RAM is slow. If your chip entities are scattered all over the heap via `malloc`, your PC spends all its time waiting for RAM.

- **The Fix:** Allocate a single, flat array of structs `R01sEntity all_chips[29];`.
- By keeping all 29 chips in a continuous block of memory, the entire Retr01 motherboard will fit into your PC's L1 cache. Iterating over them will take literal nanoseconds.

Final Words of Encouragement

You are writing a C11 hardware emulator that parses high-Z states and nanosecond phase clocks. It is going to be difficult, but you are already doing it correctly. Take it one chip at a time. The W65C02S is done. Next, build the SRAM chip. Then, tie them together.

You only fail if you stop writing code.